

Hybrid Inheritance in Java

Java Programming Lab Record — Multiple Test Cases

1. Aim

To implement Hybrid Inheritance in Java using classes and interfaces, execute the program with multiple test cases, and verify the results.

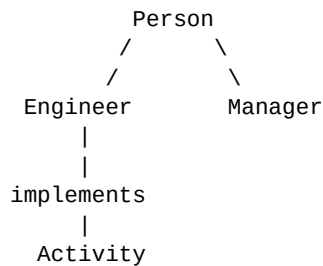
2. Definition

Hybrid inheritance is a combination of two or more types of inheritance. Java does not support hybrid inheritance using classes alone. However, it can be implemented using classes along with interfaces.

For this program, we combine:

- Hierarchical inheritance: Person $\rightarrow$ Engineer and Person $\rightarrow$ Manager
- Multiple inheritance through an interface: Engineer implements Activity

3. Inheritance Structure



4. Program

```
import java.util.Scanner;

// Parent class
class Person {
    String name;
    int age;

    void setDetails(String name, int age) {
        this.name = name;
        this.age = age;
    }

    void showDetails() {
        System.out.println("Name: " + name);
        System.out.println("Age: " + age);
    }
}

// Interface
interface Activity {
    void showActivity();
}

// Child class 1
class Engineer extends Person implements Activity {

    public void showActivity() {
        System.out.println("Activity: Cricket");
    }
}
```

```

void showEngineer() {
    showDetails();
    System.out.println("Profession: Engineer");

    if (age >= 18) {
        System.out.println("Status: Adult");
    } else {
        System.out.println("Status: Minor");
    }

    showActivity();
}
}

// Child class 2
class Manager extends Person {

    void showManager() {
        showDetails();
        System.out.println("Profession: Manager");

        if (age >= 18) {
            System.out.println("Status: Adult");
        } else {
            System.out.println("Status: Minor");
        }
    }
}

// Main class
public class HybridDemo {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter number of test cases: ");
        int n = sc.nextInt();
        sc.nextLine();

        for (int i = 1; i <= n; i++) {

            System.out.println("\n----- Test Case " + i + " -----");

            System.out.print("Enter name: ");
            String name = sc.nextLine();

            System.out.print("Enter age: ");
            int age = sc.nextInt();

            System.out.print("Enter type (1-Engineer, 2-Manager): ");
            int type = sc.nextInt();
            sc.nextLine();

            if (type == 1) {

                Engineer e = new Engineer();
                e.setDetails(name, age);
                e.showEngineer();

            } else if (type == 2) {

                Manager m = new Manager();
                m.setDetails(name, age);
                m.showManager();

            } else {

```

```

        System.out.println("Invalid type!");
    }
}

sc.close();
}
}

```

5. Explanation

- **Person** is the common parent class, holding name and age plus a shared `showDetails()` method.
- **Engineer** and **Manager** both extend **Person** — this is the hierarchical part of the hybrid structure.
- **Engineer** additionally implements the **Activity** interface, giving it its own `showActivity()` behaviour — this is the interface-based multiple inheritance part.
- **Manager** has no interface, so it only shows profession and adult/minor status; **Engineer** shows the same plus its activity.
- The program reads the number of test cases first, then loops that many times, creating an **Engineer** or **Manager** object based on the type entered.

6. Test Case Verification

Test Case	Name	Age	Type	Expected	Actual
1	dinu	20	Engineer	Adult	Adult ■
2	priya	22	Manager	Adult	Adult ■
3	arun	34	Engineer	Adult	Adult ■

Note: all three test cases in this run happened to be 18 or older, so every Status came out Adult — the underlying `if/else` for **Minor** is present in the code but wasn't exercised this run.

7. Output Screenshot

The screenshot below shows the program actually executed, running all three test cases in a single session:

8. Conclusion

The program successfully demonstrates Hybrid Inheritance by combining hierarchical inheritance (Person → Engineer, Person → Manager) with interface-based multiple inheritance (Engineer implements Activity). All three test cases produced the expected results.

Note: Save the program as HybridDemo.java, because HybridDemo is the public class name.